TYK INVENTORY & POS MANAGEMENT SYSTEM

AWS EC2 Cloud Deployment Documentation • Data Governance, Security Policy & Threat Modeling

Data Lifecycle

Explain what happens to the data in each stage of the Data Lifecycle.

Lifecycle Stage	What Happens to the Data
Creation	Cashiers, Inventory Managers, and Admins input data via the web interface (adding bicycle products/SKUs, recording stock-in shipments, and ringing up retail POS checkout transactions). System activity events and transactional records are generated automatically.
Storage	Transactional and relational business records are stored on the EC2 instance's block storage volume (Amazon EBS) within the SQLite database file (`database.sqlite`). Binary objects, generated report PDFs, and automated database backup snapshots are stored in an Amazon S3 bucket. Sensitive user credentials (passwords) are irreversibly hashed using `bcrypt` .
Usage	The Laravel backend application on Amazon EC2 queries stock quantities, executes atomic inventory decrements upon POS checkout completion, calculates cash change due, and aggregates financial analytics (Gross Revenue, COGS, Net Profit) presented on live dashboards.
Sharing	Authorized users access processed data through their role-based views. Printable customer sales invoices, POS thermal slips, and official business reports (PDF/CSV) are retrieved from Amazon S3 / local storage and transmitted securely to clients over HTTPS (TLS encryption).
Archiving	Past transaction ledgers, closed sales orders, and automated database snapshots (`.sqlite` / SQL dumps) are packaged periodically and transferred to Amazon S3 Standard-IA (Infrequent Access) or S3 Glacier using automated lifecycle rules for cost-effective disaster recovery.
Deletion	Soft-deleted records and obsolete temporary records are permanently purged from database tables using parameterized SQL queries. Expired archive objects and transient PDF exports stored in Amazon S3 are automatically removed based on designated bucket lifecycle expiration policies.

Table 1: Data Lifecycle Matrix

The Data Lifecycle Matrix shows how data moves through the system from initial input to final deletion. Data is first collected through the frontend, stored securely in the local application database (Amazon EBS) and cloud object storage (Amazon S3), processed by the backend on EC2, and accessed by authorized users. Security controls such as IAM roles, HTTPS (TLS), and restricted bucket and database access protect the data at each stage. Retention policies ensure that data is not stored indefinitely, reducing storage costs and security risks.

Overall, the system demonstrates basic cloud data governance principles, including access control, lifecycle awareness, and structured data flow. However, improvements such as automated lifecycle policies, enhanced logging, and monitoring could further strengthen security and reliability.

Policy and Access Matrix

Identify IAM users and the policies and permissions set for them with justification on why those policies and permissions were granted.

IAM Entity	Policy Attached	Justification
EC2 Backend Role (Instance Profile)	`AmazonSSMManagedInstanceCore`, Custom S3 Policy: `s3:PutObject`, `s3:GetObject`, `s3:ListBucket` (scoped to designated backup bucket)	Allows the backend application running on EC2 to securely upload, retrieve, and list files and database backups in the designated S3 bucket without hardcoding AWS credentials in `.env`. Follows the Principle of Least Privilege by restricting access to one bucket only.
DevOps / Administrator User	`AmazonEC2FullAccess`, `AmazonS3FullAccess`, `IAMReadOnlyAccess`	Required to provision and manage infrastructure components including EC2 instances, S3 buckets, security groups, and Nginx/SSL configuration. Permissions are granted strictly for deployment, maintenance, and administrative configuration.
Auditor User (Optional / Evaluator)	`ReadOnlyAccess`	Allows academic panelists and auditors to monitor and evaluate cloud architecture, instance health, S3 storage, and security groups without the ability to modify or delete infrastructure. Ensures security oversight while preventing accidental changes.

Threat Model

Using the STRIDE model, identify possible vulnerabilities in your application and deployment strategy as well as mitigations to prevent exposure of data.

Threat	Vulnerability	Justification & Mitigation
Spoofing	**Low**	If authentication is not properly implemented, attackers could impersonate legitimate cashiers or administrators. However, enforced HTTPS/TLS encryption via Certbot, bcrypt password hashing, session regeneration, and SSH key pairs (`.pem`) mitigate credential spoofing at application and infrastructure levels.
Tampering	**Low**	Data in transit could be modified if HTTPS is not enforced between client and backend. S3 objects or database records could also be altered if bucket policies are misconfigured. Proper TLS certificates, restricted IAM bucket policies, CSRF token verification, and atomic database transactions (`DB::transaction`) mitigate this risk.
Repudiation	**Low**	Without proper logging (CloudTrail, application logs, Nginx logs), users or attackers could deny performing actions (e.g., unauthorized stock adjustments, file uploads, or deletions). The application features an immutable `ActivityLog` audit trail, while Nginx and AWS system logs provide non-repudiation.
Information Disclosure	**Low**	Misconfigured S3 bucket permissions or exposed environment variables (`.env`) could leak sensitive data. Setting Nginx web root strictly to `/public`, using private S3 buckets and IAM roles instead of access keys, applying `chmod 664` to database files, and role-based field masking significantly lower exposure.
Denial of Service (DoS)	**High**	The EC2 instance (especially `t2.micro` free tier with 1 vCPU and 1 GB RAM) could be overwhelmed by high traffic or malicious requests. Without rate limiting, autoscaling, or AWS WAF, the backend is susceptible to disruption. Nginx rate limiting (`limit_req_zone`), request buffers, and pre-compiled assets (`npm run build`) mitigate server disruption.
Elevation of Privilege	**Low**	If IAM roles follow least privilege and no root credentials are exposed, privilege escalation risk is minimal. Server-side route middleware (`role:admin`) and unprivileged Linux execution (`www-data` / `ubuntu`) prevent vertical privilege escalation, while avoiding overly permissive IAM policies ensures minimal exposure.